RAYLAB

SOLUCIONES DIGITALES

Manual del proyecto y del agente de WhatsApp

Web, infraestructura, configuración, errores resueltos y operación.

Versión 1.0 · 27 de octubre de 2026 · Uso interno

Hito confirmado: respuesta automática directa en WhatsApp con el número de prueba de Meta.

El número empresarial, la memoria de conversación y la transferencia humana siguen pendientes. Sin contraseñas ni tokens en este documento.

Arquitectura: conversación automática de prueba

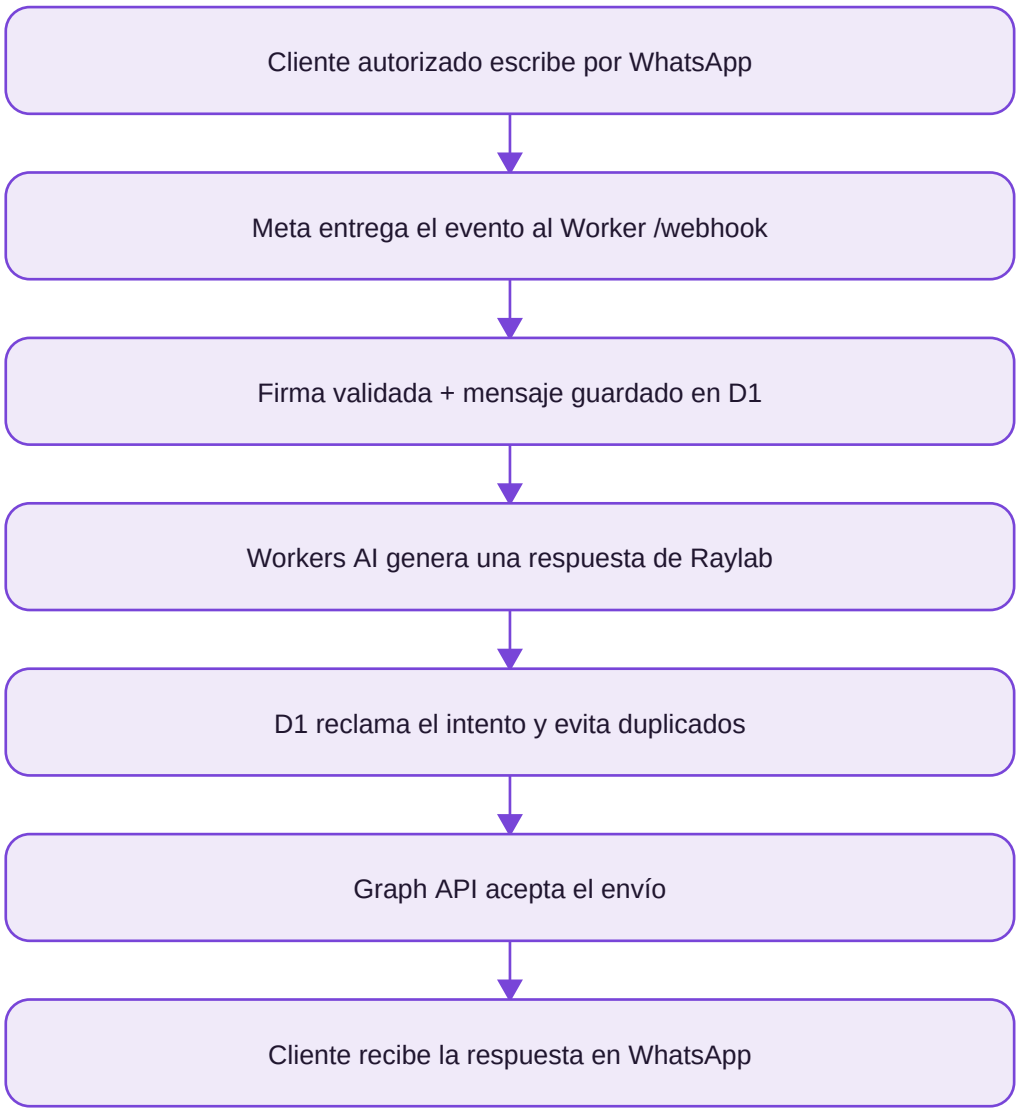
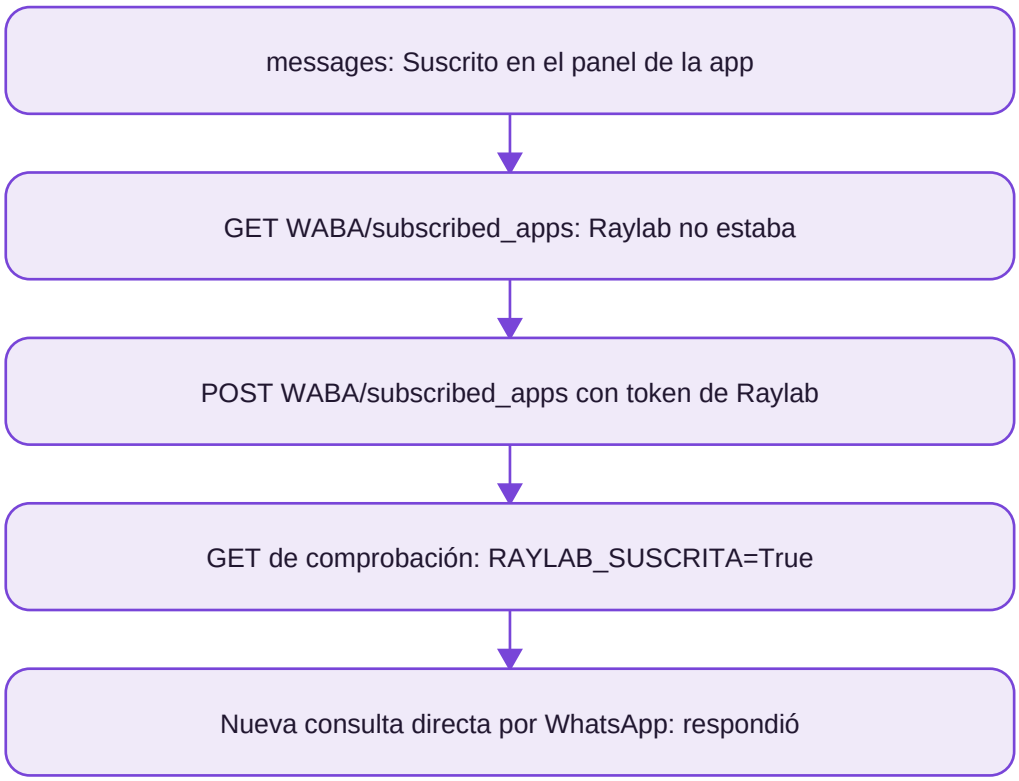


Diagrama explicativo elaborado para el manual; no es una captura del panel.

La solución del bloqueo de recepción



Son dos suscripciones distintas. Una prueba sintética correcta no comprobaba la vinculación del WABA.

Manual del proyecto y del agente de WhatsApp

Versión 1.0 · 27 de octubre de 2026 · Uso interno.

Actualización de continuidad, 28/10/2026: el propietario conectó el número empresarial +51 906 962 740 a YCloud mediante coexistencia y confirmó mensajes entrantes guardados y respuestas de IA con el destinatario de prueba. La vinculación anterior estaba en «Publicidad Asia», dentro de Cuentas de mensajes; el propietario la retiró y pudo conectar YCloud. El estado Pending no se da por resuelto solo por esta prueba exitosa. El adaptador utiliza /ycloud-webhook, firma propia y YCLOUD_API_KEY. Se preparó después una ampliación multicliente con límites, memoria breve aislada y pausa manual/por solicitud de humano; requiere migración y despliegue, aún no confirmados. Detalles operativos actuales: `agents/whatsapp-api/docs/YCLOUD.md` y `agents/whatsapp-api/docs/MULTICLIENTE.md`. El resto del manual conserva la foto histórica del primer hito del 27/10.

Responsable: Gherson Bruce Rivadeneyra Aliaga. Este documento reúne el estado del código local, las decisiones del proyecto y los resultados confirmados por el propietario en la conversación. Los secretos no se incluyen. Los resultados manuales en Meta y Cloudflare se distinguen de las pruebas automáticas locales.

Índice

1. Estado actual y alcance
2. Marca, servicios y contenido
3. Arquitectura y recursos
4. Organización de archivos
5. Edición y publicación de la web
6. Diseño móvil, logo y animaciones
7. Configuración de Meta y Cloudflare
8. Funcionamiento del agente
9. Base de datos y consultas
10. Instalación y despliegue del agente
11. Comandos de operación
12. Incidentes y soluciones
13. Seguridad, privacidad y eliminación
14. Pruebas y evidencias
15. Paso al número empresarial
16. Continuidad, límites y mejoras
17. Referencias y capturas

1. Estado actual y alcance

Resultado confirmado: el propietario escribió directamente al número de prueba de Meta y recibió la respuesta del agente sin ejecutar un comando en la terminal. La confirmación fue: «SI YA ME RESPONDIO

DIRECTAMENTE EN EL WSP».

Elemento	Estado al cierre
Web Raylab en Cloudflare Pages	Publicada por el propietario
Logo y paleta morada	Aprobados e incorporados
Páginas de privacidad y eliminación	Preparadas, subidas y URLs guardadas en Meta según el propietario
Aplicación de Meta	Publicada por el propietario
Recepción firmada y guardado D1	Verificados con muestra; conversación automática confirmada posteriormente
Workers AI	Respuesta real confirmada desde la terminal
Envío por Graph API	Aceptado por Meta y recibido en el teléfono
Suscripción de Raylab a WABA	Confirmada con consulta de Graph API
Respuesta automática en WhatsApp	Confirmada con número de prueba
Número empresarial real	Pendiente de conexión y validación de coexistencia
Memoria de conversación	No implementada
Transferencia y pausa humana	No implementadas
Cobros, agenda, CRM y ERP interno	No implementados

El agente sigue limitado al destinatario de prueba configurado. La app publicada no significa que el número empresarial esté conectado ni que el servicio esté listo para atender a todos los clientes.

2. Marca, servicios y contenido

La marca elegida es **Raylab Soluciones Digitales**. El fundador es desarrollador Odoo con más de tres años de experiencia declarada en páginas web, microapps y bases de datos. Se mantiene una presentación empresarial, sin inventar integrantes, certificaciones ni casos de éxito.

La identidad aprobada usa un símbolo de rayo con referencia visual a la letra R. El rayo representa velocidad y se relaciona con el nombre Raylab. El archivo aplicado en la web es `sites/raylab/dist/img/logo.svg`.

Color	Referencia
Morado principal	`#8B5CF6`
Lila	`#C4B5FD`
Fondo oscuro	`#100B20`

El mensaje principal de la web es **«Tecnología que impulsa tu empresa»**. Los servicios son desarrollo Odoo, diseño y desarrollo web, sistemas empresariales a medida y microapps. Se sustituyó el lenguaje «ERP sencillos»

por una propuesta más empresarial. Se puede empezar por una revisión técnica de Odoo, una landing page o una microapp enfocada en una tarea concreta.

Los presupuestos y plazos se definen por evaluación. El agente no debe inventarlos. El proceso comercial es entender la necesidad, definir alcance y propuesta, desarrollar con revisiones y validar la entrega.

La web contiene presentación, nosotros, servicios, formas de empezar, metodología, preguntas frecuentes y contacto. El formulario prepara un mensaje para WhatsApp; el visitante debe enviarlo. Se redujeron los accesos repetitivos a WhatsApp en las tarjetas. Los enlaces principales navegan por la web; se conserva el acceso flotante y el formulario de contacto.

El número público actual de la web es `+51 953791221`. No se debe sustituir por el nuevo número empresarial hasta validar su conexión. No confundirlo con el número de prueba de Meta ni con `WHATSAPP_TEST_RECIPIENT`.

3. Arquitectura y recursos

```
WEB PUBLICA
Visitante -> Cloudflare Pages -> archivos HTML / CSS / JS
Formulario -> abre WhatsApp con mensaje preparado -> visitante envia

AGENTE DE PRUEBA
WhatsApp del destinatario autorizado
  -> numero de prueba de Meta
  -> webhook HTTPS del Worker
  -> verificacion de firma y Phone Number ID
  -> guardado D1 (inbound_messages)
  -> comprobacion de destinatario, antiguedad y duplicados
  -> Workers AI genera una respuesta
  -> D1 guarda la intencion de envio (outbound_replies)
  -> Graph API envia -> WhatsApp recibe
```

La web y el agente son proyectos distintos dentro de la misma cuenta de Cloudflare. Subir el ZIP de la web no actualiza el Worker del agente.

Recurso	Valor operativo; no es una contraseña
Proyecto Pages	`rayliotechsolutions`
Web	`https://rayliotechsolutions.pages.dev/`
Worker	`raylio-whatsapp-agent`
URL del Worker	`https://raylio-whatsapp-agent.rivasbruce563.workers.dev`
Webhook	`/webhook`
Estado técnico	`/health`
Prueba privada	`/ai-test`
D1	`raylio-whatsapp-inbox`
Aplicación Meta	`Raylab Agente WSP`
App ID	`970093816170921`
Portafolio actual	`1582829903106079`

Recurso	Valor operativo; no es una contraseña
WABA / identificador de cuenta de mensajes de prueba	`1201660269199867`
Phone Number ID de prueba	`1341194582416476`
Número de teléfono de prueba	`+1 (555) 147-0523`
Modelo vigente en el código	`@cf/meta/llama-3.1-8b-instruct-fp8`

Estos IDs corresponden a esta configuración. Si Meta recrea un recurso, revisar los valores actuales del panel. Los portafolios antiguos no se usan en este flujo. Un número de teléfono, un Phone Number ID, un WABA ID, un App ID y un Business Portfolio ID son objetos distintos.

4. Organización de archivos

```
AGENCIA DIGITAL/
  README.md                      entrada del proyecto
  docs/
    RAYLAB-MANUAL.md              fuente editable de este manual
    RAYLAB-MANUAL.pdf             copia de lectura y para compartir
    generar-manual.py             regenera el PDF
  sites/raylab/
    .git/                        historial; conservar
    .openai/                     metadatos de publicacion anterior; conservar
    dist/                        web, recursos y editor local
  output/raylab/
    raylab-web-hosting.zip       paquete publico para Pages
    package-hosting.py           generador con lista cerrada
    REVISION-SEGURIDAD.md        revision historica de la web
  agents/
    raylab-whatsapp.md           base de conocimiento e instrucciones extensas
    whatsapp-api/
      worker.mjs                  codigo autonomo del Worker
      worker.test.mjs             pruebas automaticas locales
      schema.sql                 tablas de D1
      wrangler.jsonc             configuracion local; D1 aun sin ID sincronizado
      test-ai.ps1                prueba privada, no envia a WhatsApp
      test-whatsapp.ps1          prueba manual con envio real al destinatario fijo
      check-subscription.ps1 lectura de apps suscritas al WABA
      subscribe-raylab.ps1       suscribe la app del token y verifica Raylab
      README.md                  guia del backend
      WORKERS-AI.md              configuracion de IA
      WHATSAPP-SEND.md           configuracion de envio y limites
```

Los documentos y scripts privados no van en el ZIP público. No borrar `.git`, `.openai`, `_headers`, los archivos vinculados desde HTML ni los scripts de operación pensando que todo archivo adicional es basura.

5. Edición y publicación de la web

Quieres cambiar	Archivo o lugar
Textos y estructura principal	`dist/index.html`
Servicios y catálogo	`dist/js/services-data.js` o editor local `dist/admin.html`
Escenas ilustrativas y texto dinámico de la portada	`dist/js/experience.js`

Quieres cambiar	Archivo o lugar
Identidad gráfica	`dist/img/logo.svg`, `brand.css`
Estilos generales	`style.css` y hojas específicas de cada sección
Ajustes móviles	`responsive.css`
Movimiento	`motion.js`, `motion.css`, `experience.js`, `premium.js`
Número y mensajes de WhatsApp de la web	`whatsapp-core.js`; revisar también enlaces fijos en HTML
Privacidad y eliminación	`privacidad.html`, `eliminacion-datos.html`, `legal.css`
Cabeceras HTTP	`_headers`

No todo el texto que se ve en el navegador está escrito en `index.html`: JavaScript construye partes del catálogo y del panel animado. Buscar la frase con la búsqueda global de VS Code para localizar su origen.

El editor de servicios es local: los borradores se guardan en el navegador, no en un administrador remoto. Para publicar un catálogo, guardar el archivo exportado en `dist` o reemplazar `services-data.js`; conservar un respaldo de los borradores cuando corresponda.

Después de editar, desde la raíz del proyecto:

```
python tools/web/package-hosting.py
```

El generador valida que existan los 26 archivos permitidos, que no sean enlaces simbólicos, que el total sin comprimir no exceda el presupuesto local de 5 MiB y que el ZIP sea válido. Ese presupuesto es una decisión del proyecto, no una afirmación del límite de Cloudflare. Excluye los archivos `admin.*`, agentes, secretos, documentación e historial.

Publicar en Cloudflare: Workers & Pages → `rayliotechsolutions` → Create deployment → Production → subir `raylab-web-hosting.zip`. Verificar la web publicada después de cada cambio. Abrir archivos locales no actualiza el dominio.

URLs de las páginas informativas:

```
https://rayliotechsolutions.pages.dev/privacidad.html
https://rayliotechsolutions.pages.dev/eliminacion-datos.html
```

6. Diseño móvil, logo y animaciones

Se incorporaron estilos para móviles, campos de formulario legibles, navegación móvil y disposición del panel ilustrativo. Se corrigieron superposiciones entre logo, tarjeta de código y texto del panel. La composición por capas se conservó; no se pretendía convertir todos los elementos en una lista vertical en escritorio.

Las comprobaciones históricas incluyeron simulaciones de tamaños móviles en navegador. El usuario aportó capturas de un iPhone para señalar defectos. No se debe afirmar que todas las versiones de Safari, Android y todos los dispositivos físicos estén certificados.

El carrusel cambia de escena cada 5,8 segundos. Se pausa con la pestaña oculta, fuera de vista o durante navegación por teclado dentro de sus controles. Se corrigieron pausas involuntarias por hover/foco táctil y se añadió compatibilidad con APIs antiguas de MediaQueryList.

Por decisión explícita del propietario, las animaciones están activadas por defecto incluso frente a la preferencia del sistema de reducir movimiento, y el botón de control está oculto. `motion.js` elimina únicamente la preferencia antigua `raylio-motion-choice`; no borra otros datos del navegador. Esta decisión debe revisarse si se reconsidera la experiencia de accesibilidad.

Las URLs de los scripts incluyen versiones para reducir el uso de archivos viejos de caché. Si localmente funciona pero publicado no, regenerar el ZIP, comprobar el despliegue de producción y revisar qué versión de los scripts sirve el dominio antes de culpar al dispositivo.

7. Configuración de Meta y Cloudflare

Variables, secretos y bindings

Nombre exacto	Tipo	Función
`WHATSAPP_VERIFY_TOKEN`	Secret	Cadena elegida para verificar el webhook; debe coincidir en Meta
`META_APP_SECRET`	Secret	Clave secreta de la app; valida firmas entrantes
`WHATSAPP_ACCESS_TOKEN`	Secret	Identificador de acceso de Meta; autoriza Graph API
`AI_TEST_TOKEN`	Secret	Clave propia para la ruta privada `/ai-test`
`WHATSAPP_PHONE_NUMBER_ID`	Variable	ID del número emisor; no es el teléfono
`WHATSAPP_TEST_RECIPIENT`	Variable	Único destinatario autorizado, país y dígitos, sin + ni espacios
`WHATSAPP_REPLIES_ENABLED`	Variable	Texto exacto `true` para activar; `false` para detener envíos
`AI`	Binding Workers AI	Permite `env.AI.run()`
`INBOX`	Binding D1	Apunta a `raylio-whatsapp-inbox`

En el panel actual: Worker → Settings → Runtime variables and secrets → Production → Add variable. En el formulario aparecen Key, Value y una casilla Secret. Los bindings se agregan en su sección propia; no se sustituyen por variables de texto.

La app de Meta se creó con el caso de uso «Conecta con los clientes a través de WhatsApp». Desde Paso 1 Probar se obtienen los IDs y el identificador de acceso. La clave secreta está en Configuración de la aplicación → Información básica. No mostrarla en capturas.

Dos suscripciones diferentes

1. En la app, configurar la URL del webhook, verificar el token y suscribir el campo `messages`.

2. En la cuenta de WhatsApp, suscribir la aplicación mediante `/{WABA_ID}/subscribed_apps`.

La primera estaba correcta y la segunda faltaba. Una prueba sintética exitosa del campo `messages` no demuestra que la segunda vinculación esté completa.

En Meta se publicaron la política de privacidad y las instrucciones de eliminación en sus campos. El propietario publicó la app. Eso no completó por sí solo la suscripción de Raylab al WABA.

8. Funcionamiento del agente

Recepción

`GET /webhook` devuelve el desafío de Meta solo si coincide el token y el modo `subscribe`. `POST /webhook` comprueba la firma `X-Hub-Signature-256` con HMAC-SHA256 sobre el cuerpo original antes de interpretar JSON. El límite de lectura es 256 KiB reales, sin confiar solo en Content-Length.

Se aceptan el sobre completo de WhatsApp y la muestra de campo firmada del panel. Se filtra el Phone Number ID y se guardan los mensajes con SQL parametrizado. Los estados de entrega/lectura no se convierten en consultas del cliente. Si falla el guardado se devuelve 503 para permitir reintentos.

Generación y envío automático

El envío requiere `WHATSAPP_REPLIES_ENABLED=true`, configuración completa y destinatario exacto autorizado. Solo se consideran mensajes textuales con contenido y timestamp reciente; los antiguos, sintéticos sin timestamp válido y medios no disparan respuesta. El código usa un margen máximo de 23 horas para iniciar la generación dentro de la ventana de atención. Un reloj futuro superior a un minuto también se descarta.

La IA recibe las instrucciones comerciales y hasta 2000 caracteres del mensaje actual. El máximo configurado de generación es 300 tokens; el envío limita el texto a 4000 caracteres. No se carga el historial: **guardar conversaciones no equivale a que el modelo tenga memoria**.

Se reclama el ID del mensaje de forma única en `outbound_replies`; la intención se guarda antes de llamar a Meta. El envío usa Graph API v26.0, `messaging_product=whatsapp`, destinatario individual y mensaje de texto. Un `messages[0].id` válido con respuesta HTTP exitosa se registra como aceptado. La entrega final debe comprobarse por WhatsApp o mediante eventos de estado; el código actual no mantiene un seguimiento completo de estados de entrega.

Instrucciones comerciales

El agente se identifica como asistente virtual, responde profesionalmente y hace una pregunta por vez. No solicita contraseñas o códigos. No inventa clientes, certificaciones, tarifas, plazos ni estados de proyectos. No puede agendar, cobrar ni transferir realmente una conversación.

La guía extensa está en `agents/whatsapp-api/docs/INSTRUCCIONES-AGENTE.md`, pero el Worker utiliza el texto `RAYLAB_PROMPT` incorporado en `worker.mjs`. Editar solo el Markdown no cambia el comportamiento desplegado: actualizar y revisar también el prompt del Worker.

Rutas de prueba

`/ai-test` exige Authorization Bearer con `AI_TEST_TOKEN`. La prueba normal devuelve texto de IA sin enviar a WhatsApp. La opción explícita `send_to_whatsapp=true` envía solo al destinatario fijo del servidor y exige un ID de solicitud válido. No se acepta un destino arbitrario proporcionado por el cliente.

El modo diagnóstico privado puede devolver un mensaje del proveedor recortado y con valores conocidos redactados. Los logs ordinarios guardan marcadores técnicos y códigos, no el contenido de conversaciones. No publicar el token ni usar esta ruta como un chat público sin protección.

9. Base de datos y consultas

La web estática no necesita una base de datos para mostrar sus servicios. El agente sí usa D1.

Tabla	Datos
`inbound_messages`	ID único, Phone Number ID, remitente, tipo, texto, fecha de recepción
`outbound_replies`	ID entrante o de prueba manual, estado, respuesta, ID de Meta, fecha de actualización

`schema.sql` crea tablas e índice con `IF NOT EXISTS`. Su ejecución es aditiva: no vacía el historial.

```
SELECT text_body, received_at
FROM inbound_messages
ORDER BY received_at DESC LIMIT 5;

SELECT inbound_id, state, meta_message_id, updated_at
FROM outbound_replies
ORDER BY updated_at DESC LIMIT 10;
```

Las fechas se almacenan como milisegundos Unix del servidor. El timestamp de WhatsApp recibido se utiliza para decidir si responder, pero no se guarda como columna propia en este esquema.

Estado saliente	Interpretación
`generating`	El mensaje fue reclamado y se está generando la respuesta
`sending`	La intención está guardada y se va a llamar a Meta
`accepted`	Meta aceptó el envío; no equivale a entrega confirmada
`rejected`	Meta rechazó la solicitud; revisar su código
`unknown`	Falló la red o la lectura de respuesta; no se sabe si Meta envió

No borrar filas de la bandeja saliente para «forzar» reintentos sin revisar el caso: podría generar un duplicado. Si se reinicia el Worker en mitad del proceso, `generating` o `sending` pueden quedar pendientes de revisión manual.

10. Instalación y despliegue del agente

1. Crear el Worker en Workers & Pages → Create application → Start with Hello World. El proyecto web Pages ya existente se conserva.

2. Crear D1 y ejecutar el `schema.sql` completo.

3. Añadir bindings `AI` e `INBOX` y configurar variables/secretos en Production.

4. Copiar todo `worker.mjs` local a `worker.js` en Edit code → versión Latest → Deploy.

5. Configurar el callback de Meta con la URL completa terminada en `/webhook`. No usar `/health` ni solo la raíz.

6. Verificar y guardar, suscribir `messages`, configurar requisitos informativos y publicar la app cuando corresponda.

7. Comprobar la suscripción del WABA y añadir Raylab si falta, usando el token obtenido desde su propia app.

8. Probar IA privada, envío manual y finalmente conversación directa por WhatsApp.

Cloudflare Access no debe bloquear el callback público de Meta. Su autenticación la realiza el propio Worker con la firma de Meta. La ruta privada de pruebas usa otro mecanismo y no debe quedar abierta.

La configuración local `wrangler.jsonc` conserva `d1_databases: []`: aún no está sincronizada con el ID real de D1 del panel. **No ejecutar** `wrangler deploy` **a ciegas**: primero reproducir los bindings efectivos. El procedimiento verificado en esta sesión fue editar y desplegar desde el panel.

11. Comandos de operación

Ejecutar desde `C:/Users/Usuario/Documents/AGENCIA DIGITAL`. Son utilizables desde la terminal Git Bash de VS Code llamando a PowerShell.

Prueba de IA, sin WhatsApp:

```
powershell -NoProfile -ExecutionPolicy Bypass -File
"./agents/whatsapp-api/scripts/powershell/test-ai.ps1"
```

Pide `AI_TEST_TOKEN` oculto y una consulta. `ExecutionPolicy Bypass` se aplica solo al proceso iniciado; no modifica permanentemente la política del sistema.

Prueba manual con envío a WhatsApp:

```
powershell -NoProfile -ExecutionPolicy Bypass -File
"./agents/whatsapp-api/scripts/powershell/test-whatsapp.ps1"
```

Antes, escribir desde el destinatario autorizado al número de prueba para abrir la ventana de atención. Confirmar SI en el script, introducir `AI_TEST_TOKEN` y una consulta. El número de destino viene de Cloudflare. Cada ejecución genera un ID nuevo: repetir el script es un nuevo intento de envío, no un reintento del mismo mensaje.

Consultar suscripción, sin cambios:

```
powershell -NoProfile -ExecutionPolicy Bypass -File
"./agents/whatsapp-api/scripts/powershell/check-subscription.ps1"
```

Pide WABA ID y el identificador de acceso de META. Devuelve IDs de apps y `RAYLAB_SUSCRITA=True/False`. Un error de permisos no demuestra que no haya suscripción. El script advierte si hay más páginas de resultados.

Añadir la app asociada al token al WABA de prueba:

```
powershell -NoProfile -ExecutionPolicy Bypass -File
"./agents/whatsapp-api/scripts/powershell/subscribe-raylab.ps1"
```

Este comando sí modifica la suscripción. Está fijado al WABA y App ID de prueba de esta sesión. No elimina otras apps. Usar el token generado desde Raylab; Graph API suscribe la app a la que pertenece el token. El script comprueba después que Raylab aparezca.

Pruebas locales del backend:

```
node --test agents/whatsapp-api/tests/worker.test.mjs
```

Detener envíos: cambiar `WHATSAPP_REPLIES_ENABLED` a `false` y desplegar. Esto no borra datos ni desactiva la recepción; tampoco deshabilita la prueba privada que solo genera IA. Se comprueba el ajuste antes de iniciar cada envío, pero no cancela necesariamente una petición que ya estaba en curso.

12. Incidentes y soluciones

12.1 La web anterior pedía acceso a ChatGPT

Síntoma: «No tienes acceso a este sitio» en un dominio anterior de Sites. No era una contraseña del hosting propio. Se pasó a una publicación de archivos estáticos en Cloudflare Pages. No confundir una publicación privada anterior con el sitio actual.

12.2 Portafolios, permisos y restricciones de Meta

Se vieron errores de límite de portafolios, eliminación bloqueada, restricción temporal para crear/agregar una página, guardado fallido de permisos y «No hay negocios disponibles» en la creación de app. Algunas capturas mostraban buen estado del perfil pese a restricciones en acciones comerciales.

No se identificó una causa única demostrada para los bloqueos de la cuenta anterior; no atribuirlos a falta de verificación sin evidencia. Finalmente el propietario pudo crear/usar un nuevo contexto de cuenta y portafolio, y completar la app. Eso describe lo que ocurrió, no es una recomendación de crear identidades falsas ni una solución universal. Un aviso de volver a intentar en una hora no garantiza aprobación al cumplirse ese plazo.

12.3 No se puede pegar código en Cloudflare

La pantalla mostraba «Only the latest version of your Worker can be edited» y un candado. Se estaba viendo una versión histórica de solo lectura. Seleccionar Latest en el desplegable o volver al Worker y entrar de nuevo a Edit code. No cambiar permisos de Windows ni reinstalar VS Code.

12.4 Aparece Not found en la URL del Worker

La raíz `/` no tiene una página web y devuelve 404 intencionalmente. `/health` sirve como comprobación básica y `/webhook` es el callback de Meta. Es normal que la raíz no muestre una interfaz del agente. No es prueba de caída de Cloudflare.

12.5 D1 devolvía This query returned no data

La tabla existía, pero estaba vacía. El panel de Meta mostraba una muestra con `metadata.phone_number_id=123456123`; el Worker filtraba por el ID real. Se hizo una prueba temporal controlada con ese ID sintético, se agregó soporte para el formato firmado de campo y se incorporaron logs de descarte. La fila apareció finalmente con `this is a text message`. No se aisló una causa única entre todos los cambios de esa etapa.

Después de la prueba se restauró `WHATSAPP_PHONE_NUMBER_ID=1341194582416476`. Nunca dejar el ID sintético configurado para atender mensajes reales. La muestra contiene datos ficticios: su fila no prueba que llegó una consulta escrita en el teléfono.

12.6 El binding tenía nombre AGENT-AI

El código accede a `env.AI`. El binding creado se llamaba `AGENT-AI`, que no coincidía. Se reemplazó solo ese binding por Workers AI con nombre `AI`. La base de datos permaneció como `INBOX`. No eliminar D1 para corregir un nombre de binding de IA.

12.7 PowerShell bloqueaba scripts

Error: «la ejecución de scripts está deshabilitada en este sistema». Se utilizó `powershell -NoProfile -ExecutionPolicy Bypass -File ...` para esta ejecución. No fue un error de token ni del agente.

12.8 AI unavailable / provider_failure / code=unknown

El primer manejo de errores ocultaba demasiado el detalle del proveedor. Los logs repetían un marcador genérico y no permitían identificar la causa. Se añadió diagnóstico privado redactado y se reprodujo desde la terminal. No se debía seguir cambiando secretos o bindings sin evidencia.

El detalle real fue:

```
5028: @cf/meta/infne-llama-3.1-8b-instruct was deprecated on 2026-05-30.
See the model catalog for alternatives.
```

El código había usado el identificador sin sufijo `@cf/meta/llama-3.1-8b-instruct`; el proveedor resolvía a un modelo retirado. Se reemplazó por `@cf/meta/llama-3.1-8b-instruct-fp8`, se actualizaron pruebas/documentación y se volvió a desplegar. La respuesta real de IA quedó confirmada.

12.9 El Playground volvía a GLM

El propietario seleccionaba otro modelo, pero al enviar la interfaz regresaba a Z.AI / `glm-4.7-flash`. Una respuesta de ese Playground no demostraba que funcionara el modelo de nuestro Worker. No se determinó por qué la interfaz restablecía el modelo. Se dejó de usar esa pantalla como diagnóstico y se consultó el error de la ruta privada real.

12.10 Los acentos salían como Â¿ y quÃ©

La IA ya funcionaba; el problema estaba en la decodificación de Windows PowerShell 5.1 al recibir JSON sin charset explícito. El script se cambió para leer los bytes de respuesta como UTF-8 y configurar salida UTF-8. No requirió cambiar de proveedor ni regenerar tokens.

12.11 Meta aceptaba el envío pero no había respuestas automáticas

La prueba de terminal devolvía `sent_to_whatsapp=true` y el texto llegaba al teléfono. Eso verificaba el tramo saliente, no la recepción de mensajes nuevos. D1 seguía mostrando únicamente la muestra y Observability Live no registraba POST /webhook al escribir desde WhatsApp.

Se completaron las páginas informativas y se publicó la app, pero el problema persistió. El campo `messages` figuraba Suscrito. Se consultó entonces el WABA:

```
APP_ID=2202427980234937
RAYLAB_SUSCRITA=False
```

La app Raylab `970093816170921` no estaba suscrita a esa cuenta. No se atribuyó una identidad a la otra app sin verificarla. Se ejecutó POST /WABA/subscribed_apps con el token de Raylab y se consultó de nuevo:

```
META_SUCCESS=True
RAYLAB_SUSCRITA=True
```

Después de ese cambio el propietario confirmó la respuesta directa en WhatsApp. Esta fue la corrección decisiva observada de la vinculación pendiente. No era un problema del chip ni de la web.

12.12 Mensaje verde de test no equivale a guardado o entrega

«Se ha realizado el test correctamente» en Meta indica éxito de su prueba HTTP. Para verificar almacenamiento, consultar D1. `accepted_by_meta_not_delivery_confirmation` indica que la API aceptó el envío, no que el teléfono lo recibió. En esta sesión el propietario sí confirmó recepción real posteriormente.

12.13 Funcionó y horas después dejó de responder

Después del primer éxito, el propietario informó que por la tarde ya no respondía. **Causa pendiente de confirmar al cierre de este manual.** El uso de un token temporal hace razonable revisar su caducidad, pero el síntoma por sí solo no la demuestra. Revisar también el destinatario configurado, el interruptor de envío y los eventos entrantes.

Primero reproducir el envío manual con `test-whatsapp.ps1`, sin compartir claves. Si devuelve `meta_rejected` con código 190, revisar invalidez/caducidad del identificador de acceso de Meta, generar uno

vigente y actualizar únicamente `WHATSAPP_ACCESS_TOKEN` como Secret en Cloudflare; no reemplazar el App Secret ni AI_TEST_TOKEN. Si el envío manual funciona pero la conversación automática no, revisar la llegada del webhook, su filtro de número, destinatario y estado de la bandeja saliente.

Si el resultado es `unknown` o el comando se interrumpe durante el envío, no repetirlo a ciegas. Una nueva ejecución genera una solicitud nueva y podría duplicar el mensaje. Consultar D1 y el teléfono antes de decidir.

Árbol breve de diagnóstico

```
No responde en WhatsApp
  |-- No aparece POST /webhook -> revisar callback, messages y WABA subscribed_apps
  |-- POST con 401 -> revisar META_APP_SECRET y firma; no desactivar validacion
  |-- ignored_phone_number -> comparar Phone Number ID
  |-- mensaje guardado sin envio -> revisar true, destinatario, timestamp, tipo
  |-- outbox generating/sending/unknown -> revisar antes de reenviar
  |-- meta_rejected -> codigo de Meta, token, permisos, destinatario, ventana
  |-- provider_failure -> prueba IA privada y detalle redactado
```

En Observability, una fila roja con HTTP 503 puede mostrar `outcome=ok`: significa que el Worker terminó de ejecutar y devolvió un error, no que la operación comercial tuvo éxito. Abrir también el evento warn de la misma petición para ver el marcador técnico.

13. Seguridad, privacidad y eliminación

Se implementaron comprobación HMAC, consultas parametrizadas, límites de entrada, filtrado de emisor/destinatario, claves privadas, control de duplicados y caché no-store en respuestas del backend. La página tiene cabeceras CSP, nosniff, protección contra marcos y otras restricciones en `_headers`.

La revisión histórica de seguridad cubrió la web estática de esa fecha. **No constituye una auditoría completa del agente ni garantiza que el conjunto sea imposible de atacar.** No hay prueba de carga real, certificación de disponibilidad ni garantía de compatibilidad con todos los dispositivos.

Los secretos solo se guardan en el panel de Cloudflare o en archivos locales privados excluidos del control de versiones. Los scripts los piden con entrada oculta, no como argumento de shell. El token de acceso temporal de Meta puede caducar; hay que preparar credenciales adecuadas para operar el servicio empresarial y revocar las que se expongan.

La política pública informa de Meta/WhatsApp, Cloudflare, D1, IA y contacto `rayliotechsolutions@gmail.com`. Las solicitudes de eliminación son manuales. No se ha añadido un botón que borre datos automáticamente ni una tarea de limpieza periódica.

Procedimiento interno para una solicitud: revisar el correo, verificar con mínimos datos que la persona corresponde a la conversación, localizar datos entrantes y salientes relacionados, determinar excepciones aplicables, eliminar lo procedente con respaldo y revisión adecuados y comunicar el resultado. Borrar primero respuestas asociadas y luego mensajes si se elimina la conversación; no ejecutar borrados masivos ni copiar números de terceros a chats de soporte. Las copias de recuperación y proveedores tienen sus ciclos propios.

Antes de atender clientes reales: definir retención, permisos administrativos, acceso humano, manejo de reclamos y privacidad de cada integración. El uso de IA no elimina la necesidad de revisar su comportamiento comercial.

14. Pruebas y evidencias

Prueba	Evidencia / alcance
Salida de plantilla Meta	El propietario recibió Order confirmed desde el número de prueba
Muestra entrante	Meta informó envío correcto y D1 mostró `this is a text message`
IA real	Terminal devolvió una respuesta de Raylab tras cambiar a fp8
Envío manual IA → WhatsApp	API aceptó y el propietario mostró el mensaje recibido
Suscripción WABA	Lectura False, POST success y posterior lectura True
Conversación automática	Confirmación explícita del propietario después de escribir en WhatsApp
Pruebas automáticas	16 pruebas Node pasaron en el último conjunto ejecutado para la implementación de envío
Scripts PowerShell	Análisis sintáctico local de los scripts añadidos
ZIP web	26 archivos permitidos; CRC y enlaces locales verificados

Las pruebas locales usan SQLite en memoria para aproximar D1 y sustitutos de IA/transporte. Verifican configuración y autorización, entrada, firmas inválidas, duplicados, errores de almacenamiento, rechazos, concurrencia y no reenvío ante resultado ambiguo. No sustituyen pruebas reales en Meta o validaciones de calidad del modelo.

No hubo prueba masiva de 100 clientes, ensayo de coexistencia en el número real, verificación física general de iPhone/Android ni pentest externo. Documentar lo confirmado no equivale a prometer esos resultados.

15. Paso al número empresarial

Objetivo: usar el número empresarial de Raylab tanto en la app WhatsApp Business del teléfono como en la automatización. Esa combinación requiere un flujo de coexistencia compatible y comprobar elegibilidad y requisitos vigentes en Meta.

No basta cambiar `WHATSAPP_PHONE_NUMBER_ID` al azar. No eliminar la cuenta móvil, desinstalarla para migrar ni registrar directamente el número real en un flujo que lo saque de la app sin entender las consecuencias. No se ha validado aún un onboarding de coexistencia propio.

Plan de transición:

1. Confirmar el flujo oficial disponible para la app y el negocio; revisar permisos y elegibilidad.
2. Preparar credenciales duraderas y permisos de activos mínimos.
3. Incorporar el número por coexistencia y verificar sus IDs reales, cuenta y suscripción a la app.
4. Implementar memoria por conversación y pausa/derivación humana antes de atención amplia.
5. Añadir límites de gasto, volumen, manejo de errores y retención.
6. Probar texto entrante/saliente, mensajes manuales desde la app y ausencia de duplicados.
7. Actualizar por separado los enlaces públicos de la web cuando se confirme el nuevo canal.

16. Continuidad, límites y mejoras

La implementación actual procesa las respuestas durante la petición del webhook; no hay cola durable de trabajos con reintentos programados. D1 persiste los mensajes y el estado del intento, pero no recupera automáticamente todos los trabajos interrumpidos. No se promete entrega exactamente una vez: se prioriza no duplicar frente a reenviar ante un resultado incierto.

El filtro de un destinatario es adecuado para esta prueba, no para múltiples clientes. Antes de ampliarlo, diseñar autorización, límites por conversación, control de presupuesto y atención humana. El modelo puede equivocarse o producir contenido inadecuado pese al prompt.

El acceso administrativo de Cloudflare y Meta, MFA, recuperación de cuenta y facturación requieren administración del propietario; no se han auditado. Consultar cuotas actuales en los paneles: cuota gratuita de IA no significa WhatsApp ni crecimiento ilimitados. Una suscripción a un chat de IA no reemplaza las credenciales de las APIs usadas aquí.

Conservar código y documentación en control de versiones sin secretos, regenerar paquetes tras cambios web y anotar despliegues relevantes. Antes de reemplazar un Worker, revisar que corresponde al proyecto y entorno correctos. No usar un ZIP de la web como respaldo de D1: son datos y recursos separados.

17. Referencias y capturas

Fuentes técnicas utilizadas en la implementación:

• Cloudflare Workers AI bindings: https://developers.cloudflare.com/workers-ai/configuration/bindings/

• Modelo fp8: https://developers.cloudflare.com/workers-ai/models/llama-3.1-8b-instruct-fp8/

• Workers AI datos: https://developers.cloudflare.com/workers-ai/platform/data-usage/

• Workers AI precios: https://developers.cloudflare.com/workers-ai/platform/pricing/

• Colección oficial Meta: https://www.postman.com/meta/whatsapp-business-platform/documentation/wlk6lh4/whatsapp-cloud-api

• Lectura de suscripciones WABA: https://www.postman.com/meta/whatsapp-business-platform/request/tl2wk2j/get-all-subscriptions-for-a-waba

• Cloudflare Pages cabeceras: https://developers.cloudflare.com/pages/configuration/headers/

El PDF incluye el logo del proyecto y diagramas explicativos. **No incluye las capturas originales del chat:** no se encontraron como archivos accesibles en este workspace. Los textos de errores y resultados se transcriben de la conversación; los diagramas no se presentan como pantallas reales de los proveedores.

Capturas de referencia aportadas durante el trabajo: portada móvil con superposiciones, selector de versión bloqueada, D1 vacío y con muestra, binding AGENT-AI y luego AI, error de modelo retirado, terminal con respuesta de IA, envío aceptado y recibido, app publicada y campo messages suscrito. Si se archivan después, recortar correos, teléfonos de terceros, chats ajenos y todo token o secreto antes de incorporarlas a una copia compartida.

Este manual documenta la implementación y el diagnóstico observados hasta su fecha. Los paneles y requisitos de proveedores pueden cambiar; el código local y los valores efectivos del entorno desplegado deben verificarse juntos.